

Fury Ads — Google Ads API Tool Design Document

Company: TGL Solutions **Website:** www.tglsolutions.com.br **Manager account (MCC) ID:** 864-652-2223
API contact: josulima90@gmail.com **Access model:** Internal / own-use (we operate only on our own Google Ads accounts) **Document date:** 10/06/2026

1. Overview

Fury Ads is an internal backend tool used by **TGL Solutions** to (a) enforce ad-content compliance and (b) monitor campaign performance across **our own** Google Ads accounts, all linked under our manager account (MCC **864-652-2223**).

The tool is **not** offered to third parties and is **not** used to manage accounts we do not own. It is operated exclusively by our internal team.

2. Intended Users & Access

- **Who:** Internal employees only (marketing and operations team).
- **How:** A REST API on our private infrastructure. There is no public access.
- **Authentication to Google:** A single OAuth 2.0 refresh token tied to our manager account, stored as an environment secret. No end-user Google credentials are collected.

3. System Architecture

The system has three components:

Component	Responsibility
REST API (Node.js + Express)	Receives compliance events and serves reporting requests
Job queue (BullMQ + Redis)	Processes ad-pause actions asynchronously, with idempotency and exponential-backoff retries
Google Ads adapter	The only component that calls the Google Ads API; selects the target account (<code>customer_id</code>) per request

Compliance flow (write):

Internal detection system

| (webhook: POST /webhook/violation)



REST API → Job queue (BullMQ) → Google Ads adapter → Google Ads API
(pause the ad)

Reporting flow (read):

Internal dashboard

| (GET /metrics/google-ads/{customerId})



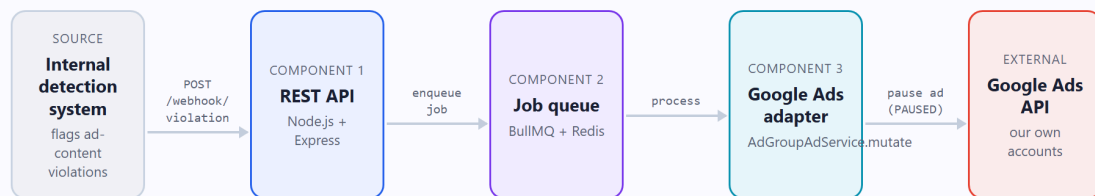
REST API → Google Ads adapter → Google Ads API (search) → JSON metrics

Fury Ads — System Architecture

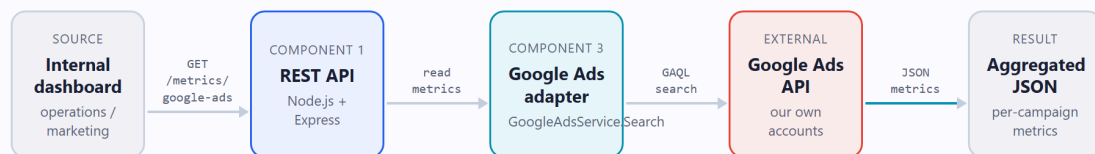
TGL Solutions · internal own-use tool · Manager account (MCC) 864-652-2223

■ Internal (own) systems ■ REST API ■ Job queue ■ Google Ads adapter ■ Google Ads API

COMPLIANCE FLOW (WRITE) — PAUSE A VIOLATING AD, ASYNCHRONOUSLY



REPORTING FLOW (READ) — READ CAMPAIGN METRICS, SYNCHRONOUSLY



THE THREE COMPONENTS

REST API

Node.js + Express. Receives compliance events (webhook) and serves reporting requests. No public access.

Job queue

BullMQ + Redis. Processes ad-pause actions asynchronously with idempotency (one job per tenant + ad) and exponential-backoff retries.

Google Ads adapter

The only component that calls the Google Ads API. Selects the target account (customer_id) per request, scoped to accounts under our MCC.

Figure 1 — System architecture: the three components and the write (compliance) and read (reporting) flows described above.

4. Google Ads API Usage

Services and operations

Service	Operation	Purpose
GoogleAdsService	SearchStream (read)	Read campaign performance metrics
AdGroupAdService	mutate (update)	Pause a violating ad (AdGroupAd.status = PAUSED)
CustomerService	ListAccessibleCustomers (read)	Enumerate the accounts we own

Reporting query (GAQL)

```
SELECT
  campaign.id,
  campaign.name,
  campaign.status,
  metrics.impressions,
  metrics.clicks,
  metrics.cost_micros,
  metrics.conversions,
  metrics.ctr,
  metrics.average_cpc
FROM campaign
WHERE segments.date BETWEEN '2025-01-01' AND '2025-12-31'
```

Mutate operation (pause an ad)

For an ad flagged by our compliance system, the adapter issues an `AdGroupAdService.mutate` update setting `AdGroupAd.status` to `PAUSED` for that ad resource. Operations are idempotent (one job per `tenantId + adId`) and retried with exponential backoff on failure.

5. Sample Request / Response

Reporting

- Request: `GET /metrics/google-ads/{customerId}?from=2025-01-01&to=2025-12-31`
- Response (one object per campaign):

```
{
  "platform": "GOOGLE_ADS",
  "customerId": "1234567890",
  "from": "2025-01-01",
  "to": "2025-12-31",
  "count": 1,
  "campaigns": [
    {
      "campaignId": "123",
      "campaignName": "Example Campaign",
      "status": "ENABLED",
      "impressions": 45200,
      "clicks": 1830,
      "costMicros": 920000000,
      "conversions": 74,
      "ctr": 0.0405,
      "averageCpcMicros": 502732
    }
  ]
}
```

Compliance (pause an ad)

- Request: `POST /webhook/violation` with `{ adId, tenantId, platform, violationType, severity, detectedAt }`
- The API responds `202 Accepted` with a `jobId` ; the job result is then available at `GET /jobs/{jobId}` as `{ jobId, status, attempts, result, error }`.

The integration is implemented and verified against our manager account: the OAuth refresh token authenticates successfully and `CustomerService.ListAccessibleCustomers` returns the accounts under MCC 864-652-2223. Live reporting/mutate calls against production accounts are pending the Basic access this application requests; until then the JSON shapes above reflect the adapter's actual response structure.

6. Data Storage & Security

- We store **only** the OAuth refresh token needed to operate on our own accounts, kept as an environment secret and never committed to source control.
- No end-user Google credentials are collected or stored.
- Every API call is scoped to a specific `customer_id` under our manager account.

7. Policy Compliance

The tool is used solely on Google Ads accounts that we own and operate. It complies with the Google Ads API Terms of Service and with the Required Minimum Functionality requirements applicable to

internal / own-use tools.